

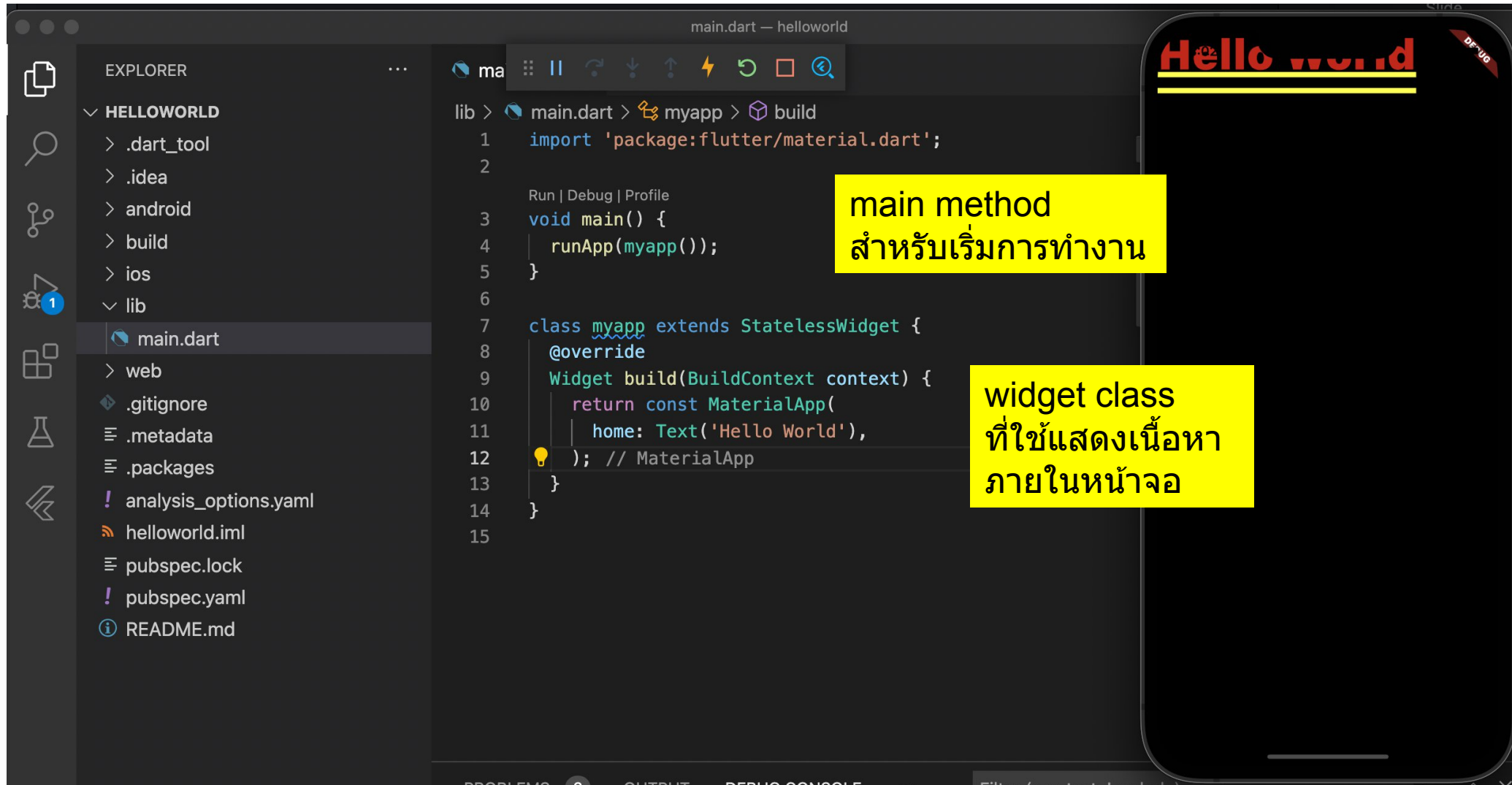
# *Flutter* Project, Structure, Assets and Layouts

By Prasertsak U., Ph.D

# Outlines

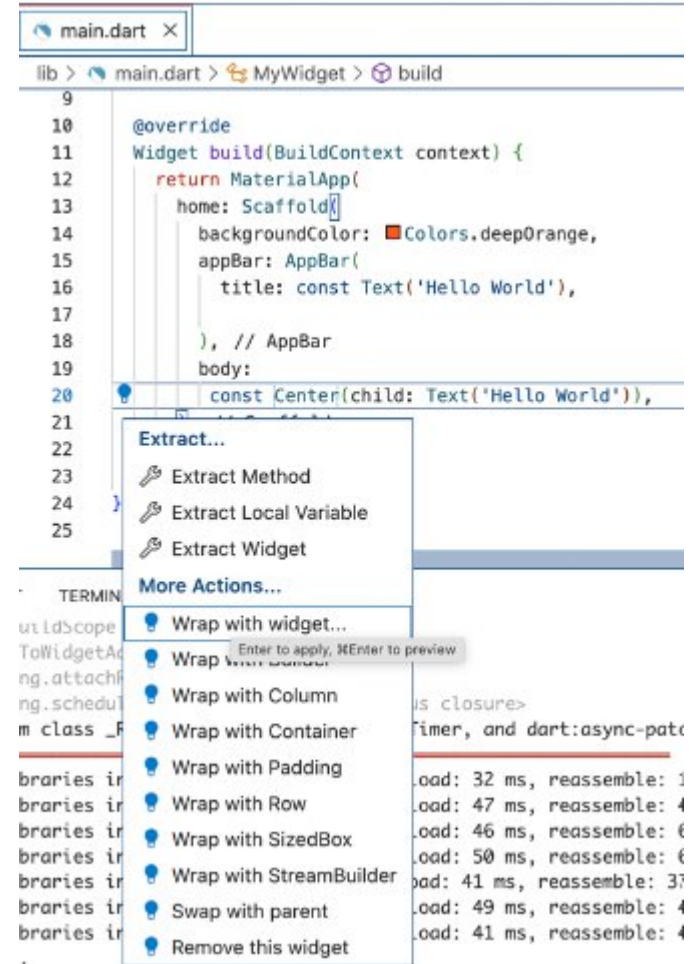
- Widgets
- Structure of Project
- Adding Assets (Font, Image file)
- Basic layouts
- Exercise

# ทำความรู้จักกับ Flutter Project

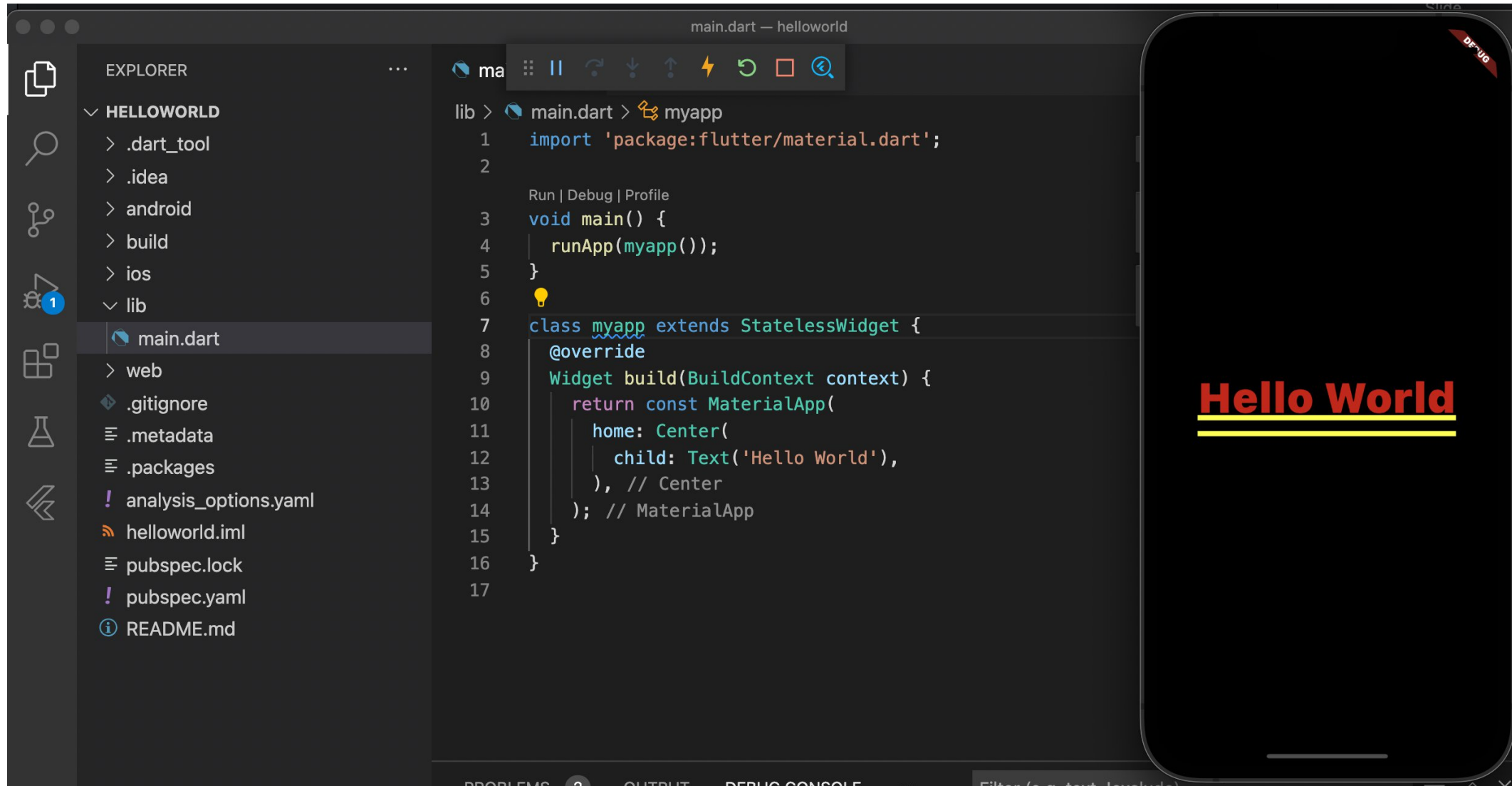


# TIPS

- การหุ้ม Widget ใด ๆ สามารถใช้ความสามารถของ IDE ช่วยได้ โดยการคลิกไปที่ชื่อของ widget ที่ต้องการหุ้ม จากนั้นให้คลิกอีกครั้งที่รูปไอคอนรูปหลอดไฟ ซึ่งจะปรากฏชื่อของ widget จำนวนหนึ่งให้เลือกได้ตามต้องการ แต่หากต้องการเลือกชื่ออื่นที่ไม่ปรากฏให้เลือกที่ *Wrap with widget* หลังจากเลือกแล้ว เราจะต้องใส่ชื่อของ Widget ที่ต้องการด้วยตนเอง
- การหุ้มด้วยวิธีนี้จะไม่ต้องทำการใส่เครื่องหมายวงเล็บครอบ หรือกำหนด child ด้วยตนเอง โดยระบบจะเพิ่มและขยับให้โดยอัตโนมัติ จะช่วยลดการเกิด syntax error ได้อย่างมาก
- วิธีนี้สามารถยกเลิกการหุ้มได้ด้วยเช่นเดียวกัน โดยคลิกที่ Widget ที่ต้องการลบออก เมื่อคลิกที่ไอคอนรูปหลอดไฟให้เลือก *Remove this widget* จากนั้น Code จะถูกแก้ไขโดยอัตโนมัติ โดยเราไม่ต้องมาปรับแก้เอง



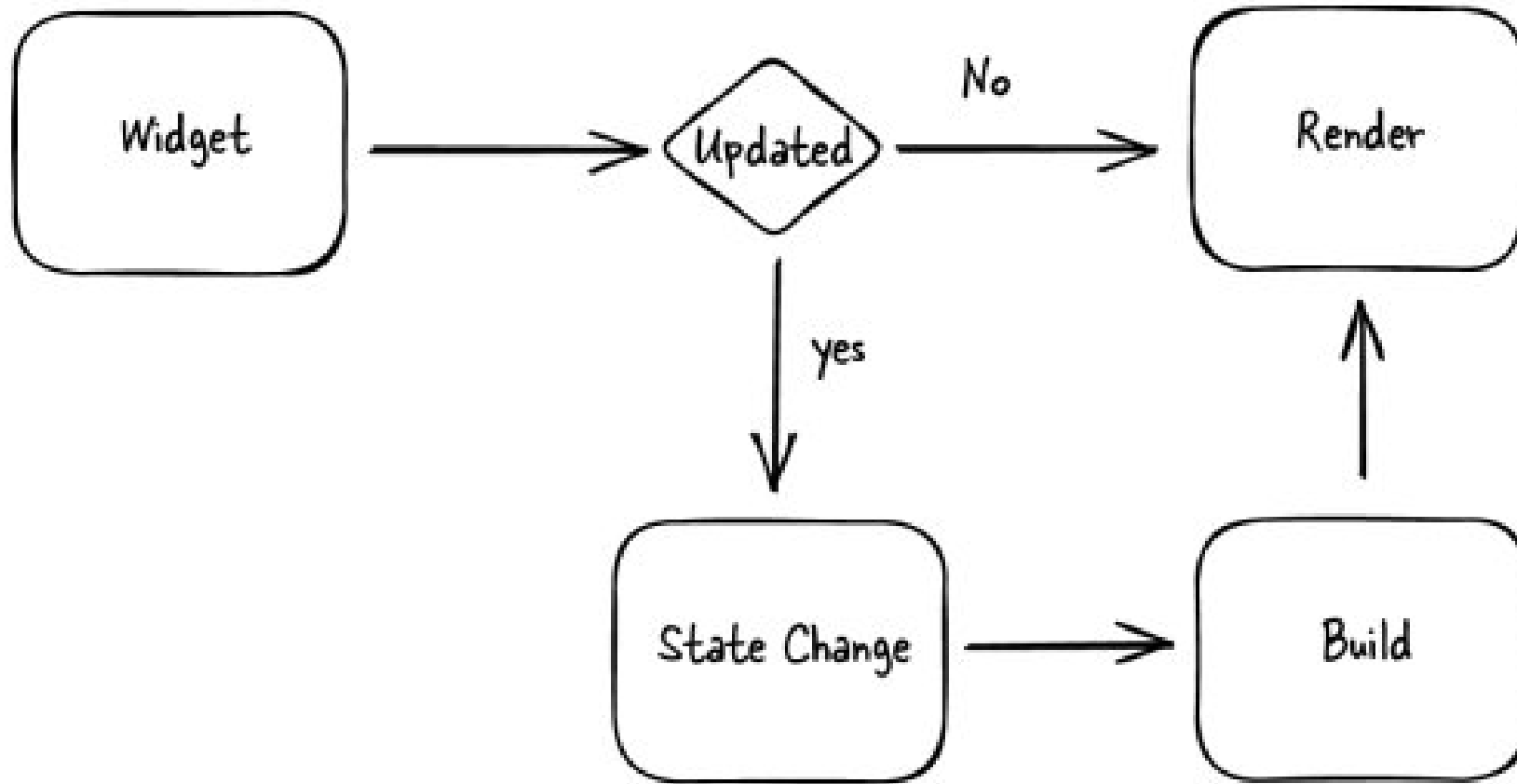
# ทำความเข้าใจกับ Flutter Project



# What is Widgets?

- Widget คือองค์ประกอบพื้นฐานที่สุดที่ใช้สร้างหน้าจอแอปพลิเคชัน (UI) โดยทุกสิ่งที่แสดงผลบนหน้าจอ เช่น ปุ่ม ข้อความ หรือการจัดวาง Layout เป็น Widget ทั้งสิ้น
- Widget จะถูกจัดวางซ้อนกันเป็น Parent และ Child เพื่อสร้างหน้าจอที่ซับซ้อนขึ้น ซึ่งเรียกการซ้อนกันเป็นชั้นๆ ของ Widget นี้ว่า โครงสร้างต้นไม้ หรือ Widget Tree
- Widget สามารถแบ่งหลักๆ ได้ 2 ชนิด ได้แก่ **StatelessWidget** (ไม่เปลี่ยนแปลงค่า) และ **StatefulWidget** (เปลี่ยนแปลงค่าได้บนหน้าจอในระหว่างทำงาน)

# Widget Rendering Loop



# ตัวอย่าง Widgets พื้นฐาน

- **Text**: เป็น Widget ที่ใช้แสดงข้อความบนหน้าจอ
- **Container**: เป็น Widget ที่เปรียบเหมือนกล่องสำหรับใส่ Widget อื่นๆ สามารถจัดรูปแบบต่างๆ เพิ่มเติมได้ เช่น สี, ระยะขอบ, ขนาด เป็นต้น
- **Row / Column**: เป็น Widget ที่ใช้เพื่อจัดเรียง Widget อื่นๆ แบบแนวนอนเรียงต่อกัน หรือแนวตั้งวางซ้อนกัน เป็นต้น
- **Scaffold**: เป็น Widget ที่ใช้เป็นโครงสร้างหลักของหน้าจอมาตรฐาน ที่มีทั้งส่วนของ AppBar, Drawer, SnackBar มาให้
- นอกจากนี้ยังมี Widget อื่นๆ ที่ใช้ในการจัด Layout เช่น **Padding**, **Center**, **Align** เพื่อกำหนดตำแหน่งในการแสดงผลให้สวยงาม เป็นต้น



# Summary type of widget

| Stateless widget                                                                              | Stateful widget                                                                     |
|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Only updates when it is initialized                                                           | Changes dynamically                                                                 |
| Text, icons, and <code>RaisedButtons</code>                                                   | Checkboxes, radio buttons, and sliders                                              |
| Does not have a <code>setState()</code> . It will be rendered once and will not update itself | Has an internal <code>setState()</code> and can re-render if the input data changes |
| Static widget                                                                                 | Dynamic widget                                                                      |
| Can't update during runtime unless an external event occurs                                   | Can update during runtime based on user action or data changes                      |

# โครงสร้างของ Project

- เปิด VS-Code และสร้างโปรเจกต์ใหม่ชื่อว่า **helloworld**

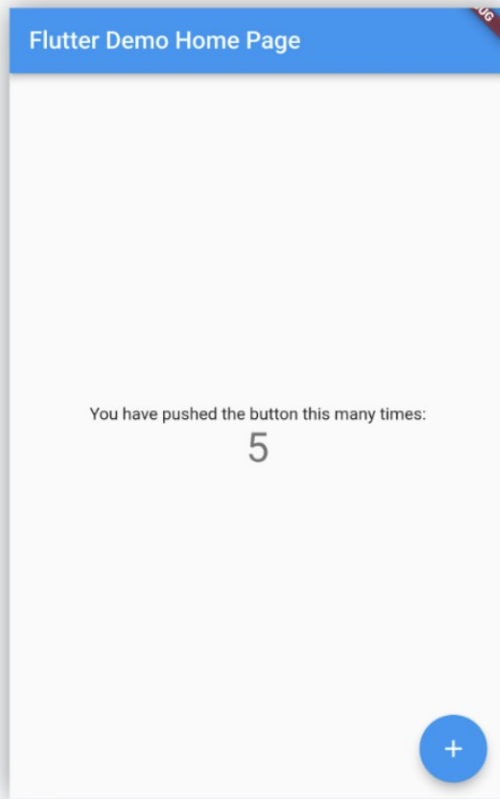
```
lib > main.dart > _MyHomePageState > build
1  import 'package:flutter/material.dart';
2
3  Run | Debug | Profile
4  void main() {
5    runApp(const MyApp());
6  }
7
8  class MyApp extends StatelessWidget {
9    const MyApp({Key? key}) : super(key: key);
10
11    // This widget is the root of your application.
12    @override
13    Widget build(BuildContext context) {
14      return MaterialApp(
15        title: 'Flutter Demo',
16        theme: ThemeData(
17          primarySwatch: Colors.blue,
18        ), // ThemeData
19        home: const MyHomePage(title: 'Flutter Demo Home Page'),
20      ); // MaterialApp
21    }
22  }
```

ไฟล์ **main.dart** มักใช้เป็นชื่อไฟล์หลัก  
สำหรับเริ่มการทำงาน  
ภายในจะมี main method เพื่อใช้ในการเริ่มการทำงาน

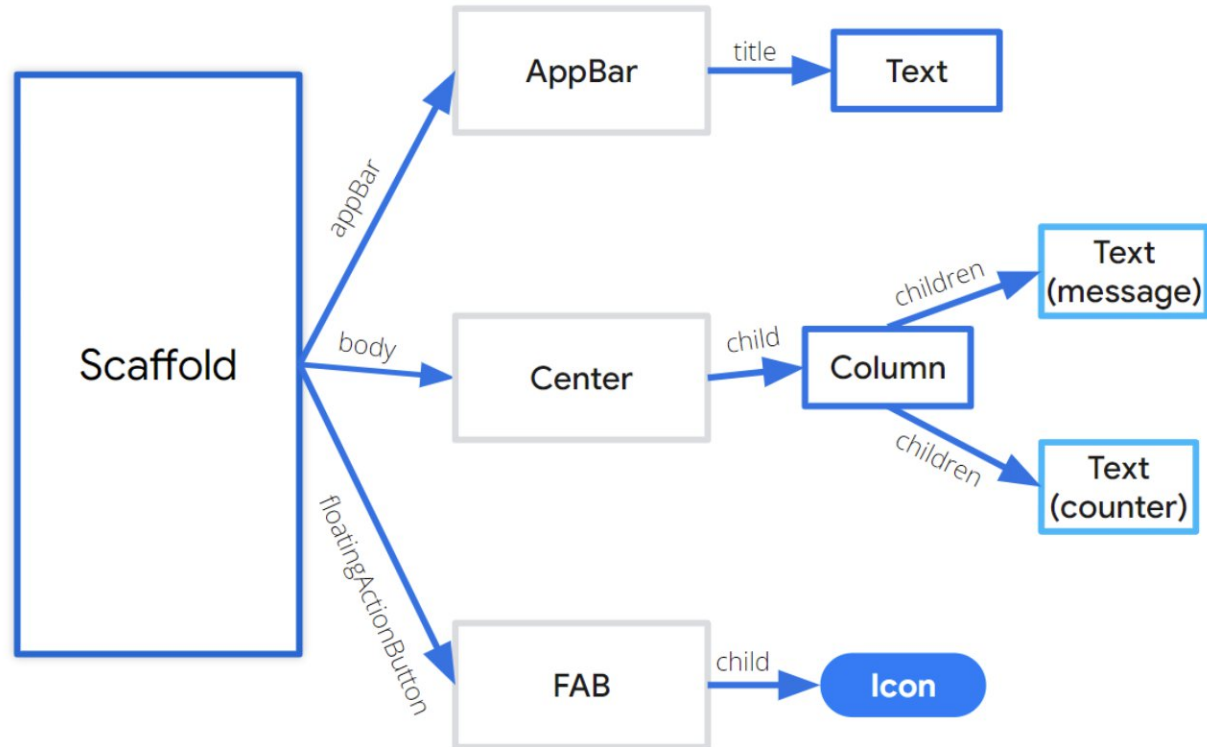
ไฟล์ **pubspec.yaml** เป็นไฟล์ที่ใช้ระบุรายการอ้างอิงทรัพยากรต่างๆ  
ที่จะนำมาใช้กับ Project เช่น ชื่อของ library ต่าง ๆ ที่นำมาใช้,  
ไฟล์ภาพ และไฟล์ Fonts ต่างๆ เป็นต้น

# โครงสร้างของ Widget ภายใน HelloWorld

UI

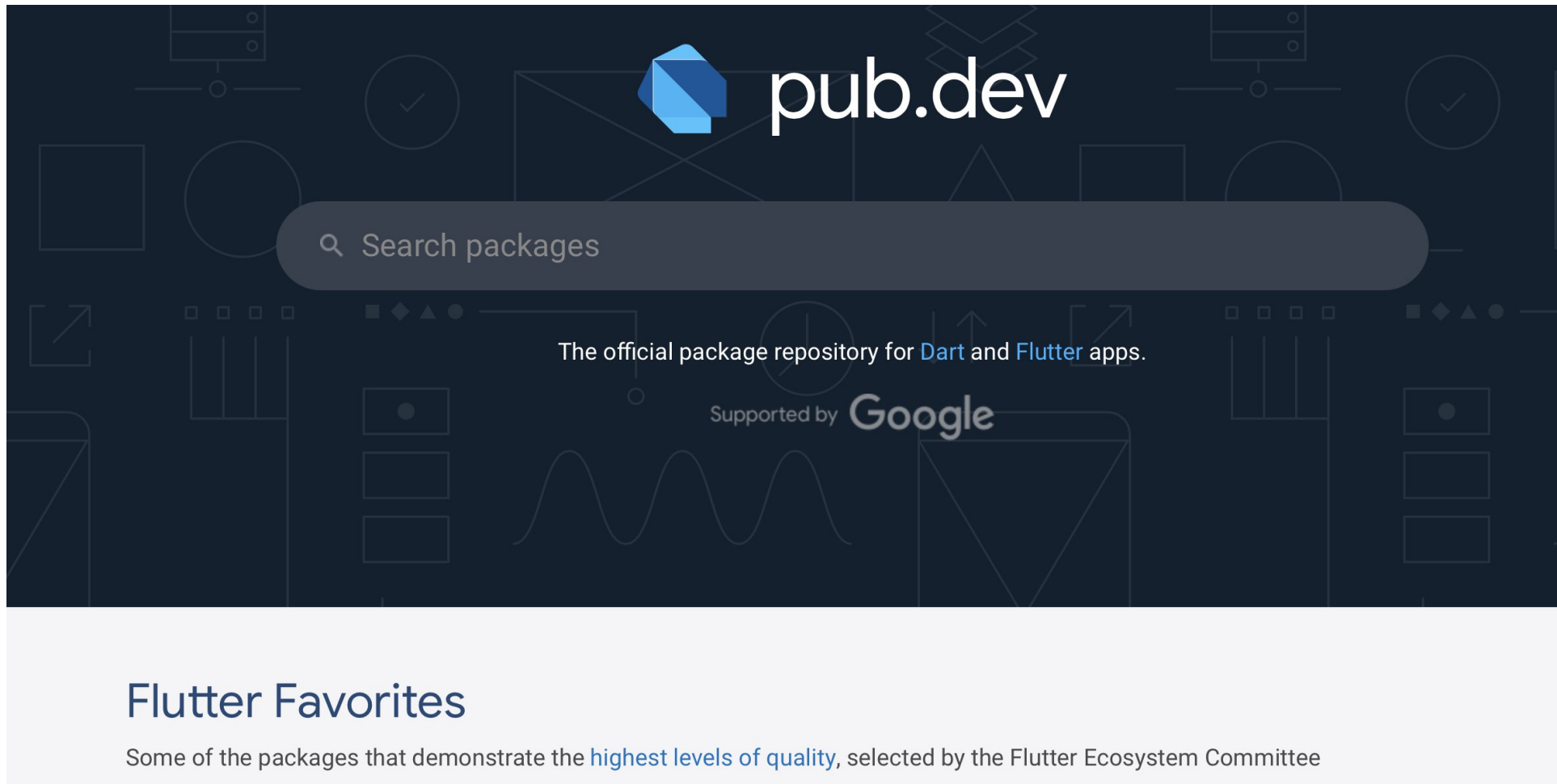


Breakdown



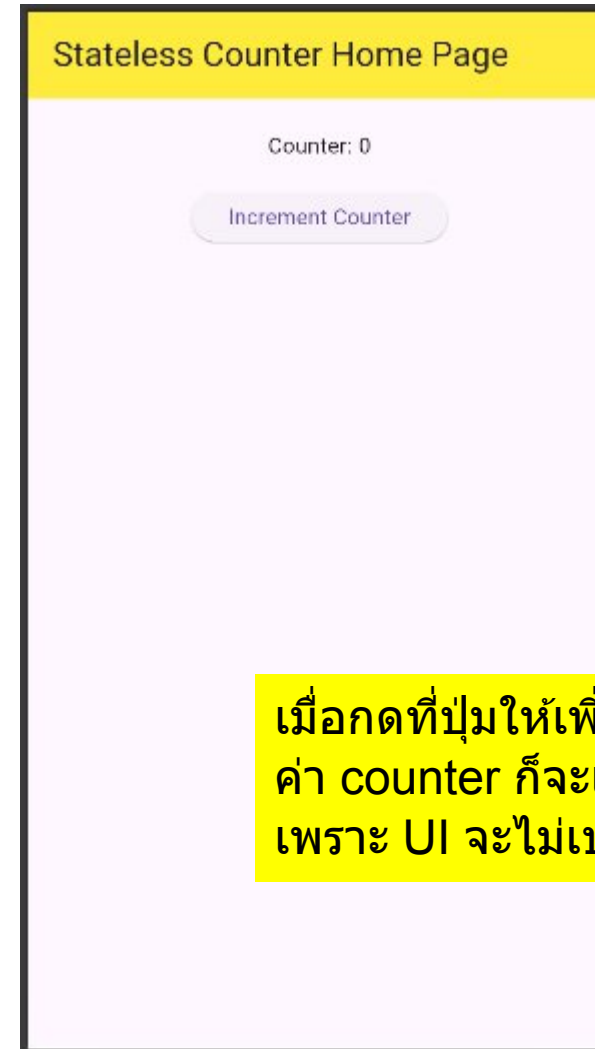
# Flutter Library Repository

- แหล่งค้นหา Flutter Library ไปที่ <https://pub.dev>



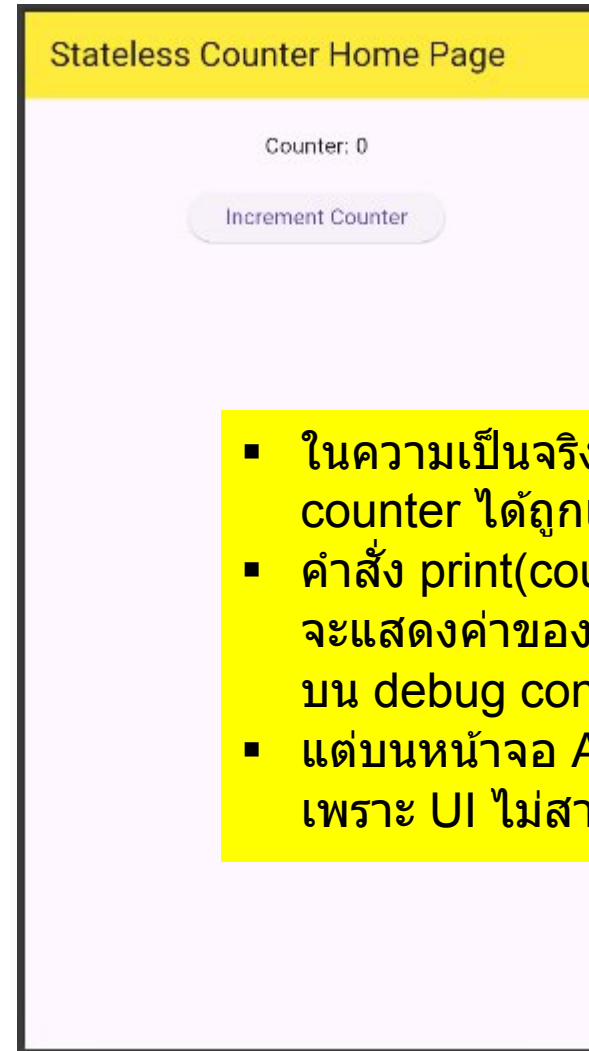
# เปรียบเทียบ *Stateless* vs *Stateful* widget

- สร้าง Project ใหม่ชื่อว่า *demo\_app*
- ให้ download โค้ดตัวอย่างจาก [ที่นี่](#) นำไปแทนที่โค้ดในไฟล์ main.dart
- โปรแกรมตัวอย่างเป็น โปรแกรมแบบ Stateless ซึ่งค่า Counter จะไม่ถูกแสดงด้วยค่าปัจจุบัน เนื่องจากการใช้ Widget Class แบบ Stateless นั้นจะไม่สามารถอัปเดต UI ได้ ในขณะที่โปรแกรมทำงาน



# เปรียบเทียบ *Stateless* vs *Stateful* widget (cont.)

```
21 class MyHomePage extends StatelessWidget {
22   int counter = 0;
23
24   void incrementCounter() {
25     counter++;
26     print(counter); // debugging purpose to see the counter
27   }
28
29   @override
30   Widget build(BuildContext context) {
31     return Scaffold(
32       appBar: AppBar(
33         title: Text('Stateless Counter Home Page'),
34         backgroundColor: Colors.yellow,
35       ), // AppBar
36       body: Center(
37         child: Column(
38           children: [
39             SizedBox(height: 20),
40             Text('Counter: $counter'),
41             SizedBox(height: 20),
42             ElevatedButton(
43               onPressed: () {
44                 incrementCounter();
45               },
46               child: Text('Increment Counter'),
47             ), // ElevatedButton
48           ],
49         ), // Column
50       ), // Center
51     ); // Scaffold
52   }
53 }
```



- ในความเป็นจริงค่าของตัวแปร counter ได้ถูกเพิ่มค่าไปแล้ว
- คำสั่ง print(counter) จะแสดงค่าของ counter ในปัจจุบันบน debug console
- แต่บนหน้าจอ App จะยังคงเป็น 0 เพราะ UI ไม่สามารถ Update ได้



# เปรียบเทียบ *Stateless* vs *Stateful* widget (cont.)

เมื่อเปลี่ยนไปใช้ Widget Class แบบ Stateful

```
21 class MyHomePage extends StatefulWidget {
22   @override
23   State<MyHomePage> createState() => _MyHomePageState();
24 }
25
26 class _MyHomePageState extends State<MyHomePage> {
27   int counter = 0;
28
29   void incrementCounter() {
30     setState(() {
31       counter++;
32     });
33     print(counter); // debugging purpose to see the counter
34   }
35
36   @override
37   Widget build(BuildContext context) {
38     return Scaffold(
39       appBar: AppBar(
40         title: Text('Stateful Counter Home Page'),
41         backgroundColor: Colors.yellow,
42       ), // AppBar
43       body: Center(
44         child: Column( // Column ...
45           // Center
46         ), // Center
47       ); // Scaffold
48   }
49 }
```

Download code [ที่นี่](#)

## Stateful Counter Home Page

Counter: 9

Increment Counter

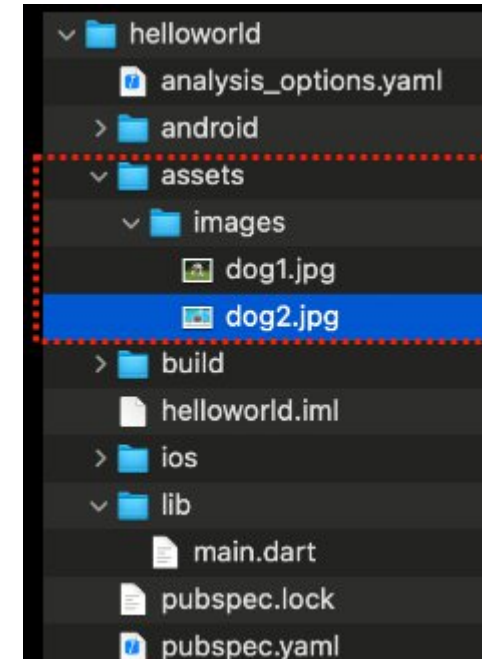
- เมื่อกดปุ่มเพิ่มค่า counter เราจะสั่งให้อัปเดต state ใหม่โดยใช้คำสั่ง `setState()` โดยภายในฟังก์ชัน `setState()` จะทำการเพิ่มค่าตัวแปร counter จากนั้น UI จะอัปเดตใหม่ด้วยค่าของ counter ที่เปลี่ยนแปลง
- คำสั่ง `setState()` จะ Error เมื่อใช้ภายใน Class ที่เป็น Stateless

# Adding Assets (images)

- Create an assets folder - การจัดการภาพใน Flutter มักนิยมสร้างโฟลเดอร์ assets ไว้ที่ Root ของโปรเจกต์เพื่อให้จัดการง่าย เช่น สร้างโฟลเดอร์ชื่อ assets/images ไว้ที่นอกสุดของโปรเจกต์ (ระดับเดียวกับโฟลเดอร์ lib)
- นำไฟล์ภาพ (เช่น .png, .jpg, .svg) ไปวางไว้ในโฟลเดอร์นั้น
- Reference an image - ใช้ไฟล์ **pubspec.yaml** เป็นหัวใจหลักในการจัดการอ้างอิงไฟล์เหล่านั้น

ตัวอย่างการใช้งานไฟล์ภาพ

```
Image.asset('assets/images/my_logo.png')
```



```
flutter:  
  uses-material-design: true  
  
assets:  
  - assets/images/my_logo.png  
  # หรือถ้าต้องการดึงทั้งโฟลเดอร์ให้ใช้:  
  # - assets/images/
```

\* หากโปรแกรมนั้นค้างไว้อยู่ และมีการแก้ไขไฟล์ pubspec.yaml ควร stop และรันใหม่ทุกครั้ง



# Adding Assets (fonts)

- ในกรณีที่มีไฟล์ Fonts อยู่แล้ว เช่น เช่น ฟอนต์จาก Google Fonts แบบไฟล์ .ttf หรือ .otf มักนิยมสร้างโฟลเดอร์ assets/fonts ไว้ที่ Root ของโปรเจกต์ (ระดับเดียวกับโฟลเดอร์ lib)
- นำไฟล์ Fonts ไปวางไว้ในโฟลเดอร์นั้น
- Reference Fonts - ใช้ไฟล์ [pubspec.yaml](https://pubspec.yaml) เป็นหัวใจหลักในการจัดการอ้างอิงไฟล์เหล่านั้น

```
flutter:  
  fonts:  
    - family: MyCustomFont # ชื่อที่เราจะเรียกใช้ใน Code  
      fonts:  
        - asset: assets/fonts/Regular.ttf  
        - asset: assets/fonts/Bold.ttf  
          weight: 700
```

ใช้งาน Font เฉพาะจุด

```
Text(  
  'สวัสดีชาวโลก',  
  style: TextStyle(family: 'MyCustomFont'),  
)
```

ใช้งาน Font ทั่วทั้ง Project

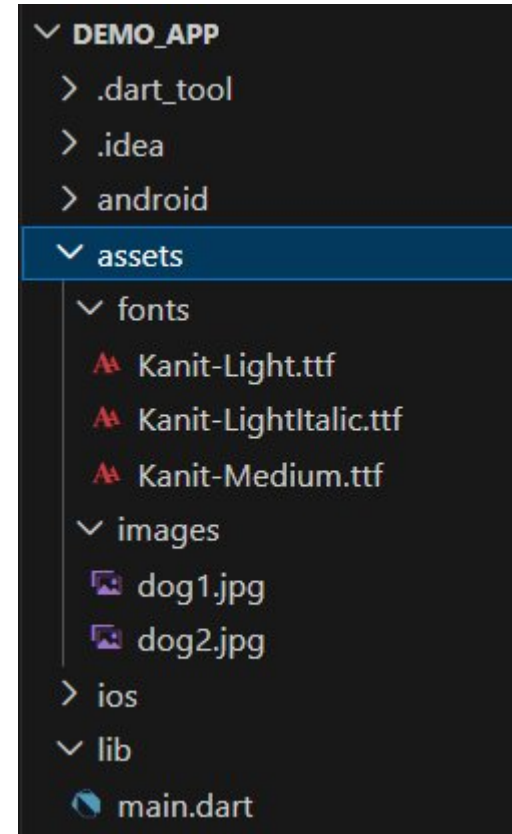
```
MaterialApp(  
  theme: ThemeData(  
    fontFamily: 'MyCustomFont',  
  ),  
  home: MyHomePage(),  
)
```

\* หากโปรแกรมนรันค้างไว้อยู่ และมีการแก้ไขไฟล์ pubspec.yaml ควร stop และรันใหม่ทุกครั้ง

# ทดสอบการใช้งาน Assets

- ภายในโปรเจกต์ **demo\_app** ให้สร้าง Folder ชื่อว่า **assets/images** และ **assets/fonts** ไว้ภายใน root ของโปรเจกต์ ให้อยู่ระดับเดียวกันกับ /lib
- นำไฟล์ภาพ และ Fonts มาวางไว้ใน Folder ที่สร้างขึ้น (ไฟล์ assets ตัวอย่างนำมาจาก [ที่นี่](#) )

ตำแหน่งการสร้าง folder



# ทดสอบการใช้งาน Assets (cont.)

- อ้างอิงถึงไฟล์ภาพ และ Fonts ทั้งหมด โดยการระบุไว้ในไฟล์ **pubspec.yaml**

```
53  flutter:
54
55  # The following line ensures that the Material Icons font is
56  # included with your application, so that you can use the icons in
57  # the material Icons class.
58  uses-material-design: true
59
60  # To add assets to your application, add an assets section, like this:
61  assets:
62    - assets/images/
63
64  fonts:
65    - family: Kanit
66      fonts:
67        - asset: assets/fonts/Kanit-Light.ttf
68        - asset: assets/fonts/Kanit-LightItalic.ttf
69          style: italic
70        - asset: assets/fonts/Kanit-Medium.ttf
71          weight: 700
72
```



เลือกวิธีการระบุทั้ง folder (ทุกภาพภายใน folder images จะสามารถใช้ได้)

# ทดสอบการใช้งาน Assets (cont.)

- แก้ไขไฟล์ main.dart ของ demo\_app โดยเพิ่มเติมการอ้างอิงถึงภาพ และ Font จาก โฟลเดอร์ assets

```
36  @override
37  Widget build(BuildContext context) {
38    return Scaffold(
39      appBar: AppBar(
40        title: Text('Stateful Counter Home Page'),
41        backgroundColor: Colors.yellow,
42      ),
43      body: Center(
44        child: Column(
45          children: [
46            SizedBox(height: 20),
47            Text('Counter: $counter'),
48            SizedBox(height: 20),
49            ElevatedButton(
50              onPressed: () {
51                incrementCounter();
52              },
53              child: Text('Increment Counter'),
54            ),
55            SizedBox(height: 20),
56            Image.asset('assets/images/dog1.jpg', width: 240, height: 240),
57            Padding(
58              padding: const EdgeInsets.all(15.0),
59              child: Text(
60                'This is a dog image referenced from assets/images/dog1.jpg',
61                style: TextStyle(
62                  fontSize: 16,
63                  fontFamily: 'Kanit',
64                  fontStyle: FontStyle.italic,
65                ),
66              ),
67            ),
68          ],
69        ),
70      ),
71    );
72  }
```

## Stateful Counter Home Page

Counter: 0

Increment Counter

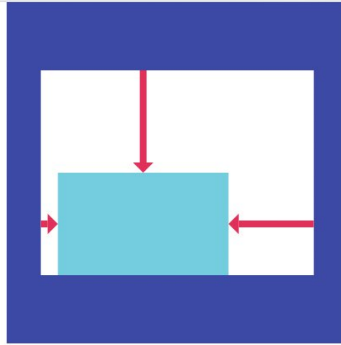


*This is a dog image referenced from assets/  
images/dog1.jpg*

# Basic Layouts

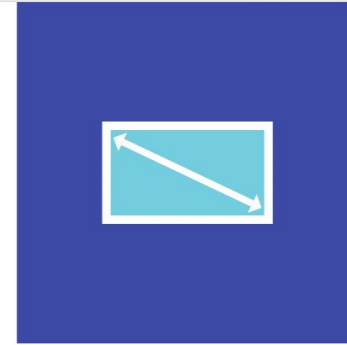
- **Single-child layouts**

- เป็น widgets ที่ทำหน้าที่จัดวางตำแหน่ง (Layout) หรือกำหนดขนาดให้แก่วิดเจ็ตลูก (Child) ได้เพียงตัวเดียว
- ภายใต้ widgets เหล่านี้จะมี widget ที่ถูกครอบอยู่ได้เพียง 1 ตัวเท่านั้น



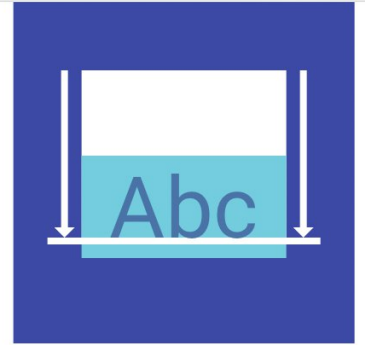
Align

A widget that aligns its child within itself and optionally sizes itself based on the child's size.



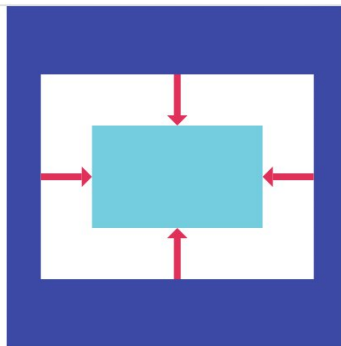
AspectRatio

A widget that attempts to size the child to a specific aspect ratio.



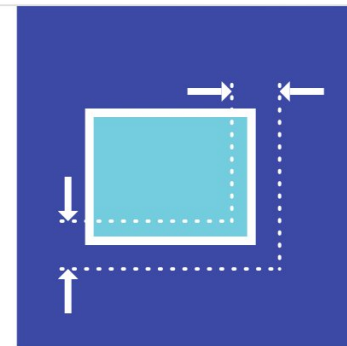
Baseline

A widget that positions its child according to the child's baseline.



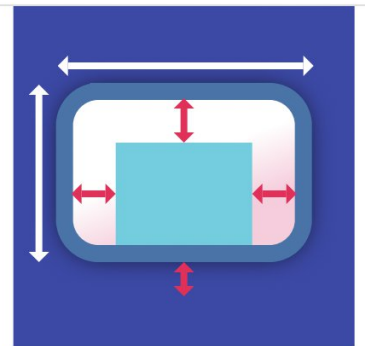
Center

A widget that centers its child within itself.



ConstrainedBox

A widget that imposes additional constraints on its child.



Container

A convenience widget that combines common painting, positioning, and sizing widgets.

# Basic Layouts

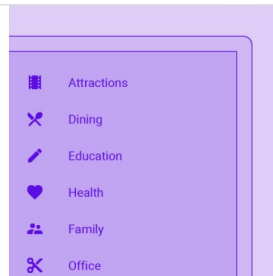
- ตัวอย่าง **Single-child Layout Widgets** ที่นิยม:
- **Container**: Widget สารพัดประโยชน์ที่ใช้กำหนดขนาด (Width/Height), สีพื้นหลัง, เส้นขอบ (Border), และระยะห่าง (Padding/Margin) ให้กับ Widget ลูก
- **Center**: ใช้สำหรับจัดวาง Widget ลูกให้อยู่กึ่งกลางของพื้นที่ที่ Widget นี้ครอบคลุมอยู่
- **Align**: ใช้จัดวางตำแหน่งของ Widget ลูกตามจุดที่ต้องการ (เช่น บนขวา, ล่างซ้าย) ภายในพื้นที่ว่าง
- **Padding**: ใช้สำหรับสร้างระยะห่างรอบๆ Widget ลูก
- **SizedBox**: ใช้กำหนดขนาดที่แน่นอน (กว้าง/สูง) ให้กับ Widget ลูก หรือใช้สร้างพื้นที่ว่างเปล่า
- **SingleChildScrollView**: ใช้สำหรับทำให้ Widget ลูกตัวเดียว (ซึ่งมักจะเป็น Column) สามารถเลื่อน (Scroll) ได้เมื่อมีเนื้อหายาวเกินหน้าจอ
- **AspectRatio**: ใช้บังคับให้ Widget ลูกมีอัตราส่วนความกว้างต่อความสูงตามที่กำหนด



# Basic Layouts

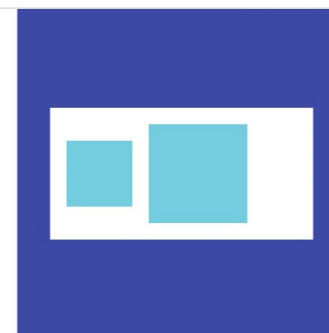
- **Multiple-child layouts**

- เป็น widgets ที่ทำหน้าที่จัดวางตำแหน่ง (Layout) หรือกำหนดขนาดให้แก่วิดเจ็ตลูก (Child) ได้หลายตัว
- โดย Widget กลุ่มนี้จะมี Property สำคัญที่ชื่อว่า children ซึ่งรับค่าเป็น List ที่ใช้ระบุ widget ลูกแต่ละตัว



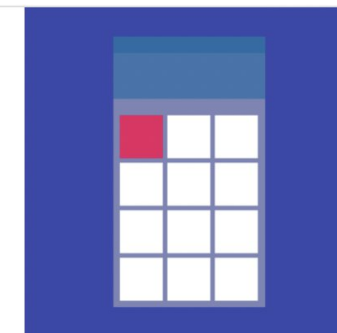
ListView

A scrollable, linear list of widgets. ListView is the most commonly used scrolling widget. It displays its children one after another in the scroll direction....



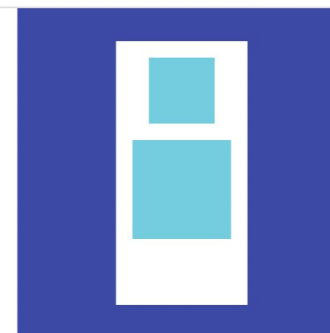
Row

Layout a list of child widgets in the horizontal direction.



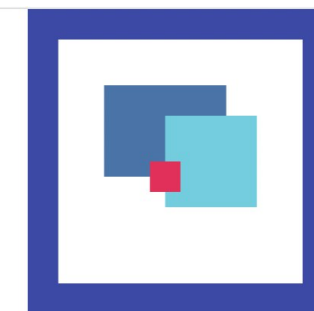
GridView

A grid list consists of a repeated pattern of cells arrayed in a vertical and horizontal layout. The GridView widget implements this component.



Column

Layout a list of child widgets in the vertical direction.



Stack

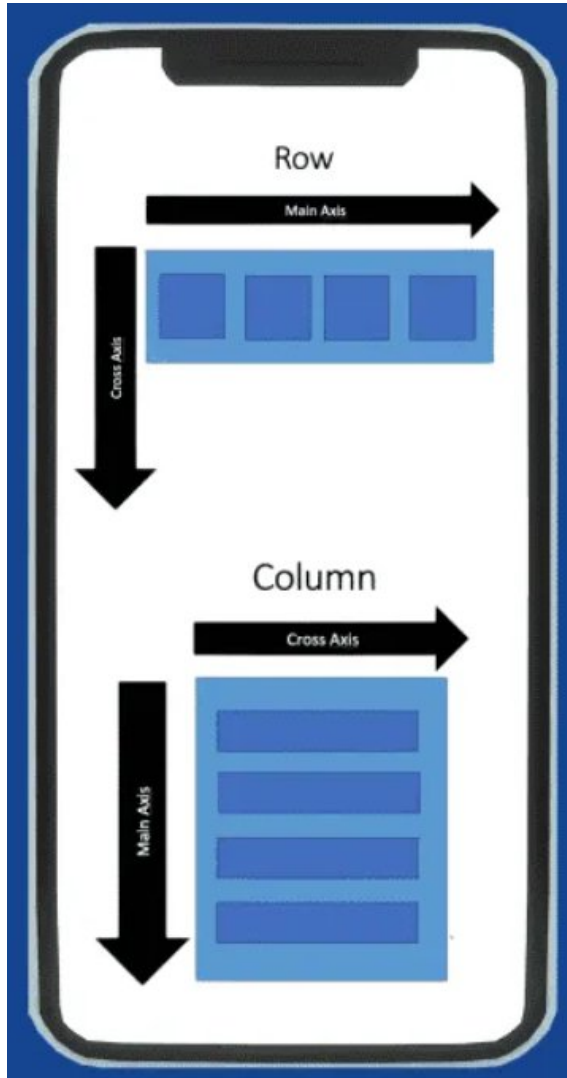
This class is useful if you want to overlap several children in a simple way, for example having some text and an image, overlaid with...

# Basic Layouts

- ตัวอย่าง **Multiple-child Layout Widgets** ที่นิยม:
- **Row**: จัดวาง Widget ลูก ที่จัดเรียงกันในแนวนอน (ซ้ายไปขวา)
- **Column**: จัดวาง Widget ลูกเรียงกันในแนวตั้ง (บนลงล่าง)
- **Stack**: จัดวาง Widget ลูกแบบซ้อนทับกัน (เหมือนการวางแผ่นกระดาษซ้อนกัน) โดยตัวที่อยู่ลำดับหลังใน List จะทับตัวก่อนหน้า
- **ListView**: จัดวาง Widget ลูกเรียงกันในแนวตั้งและทำให้ เลื่อนดูได้ (Scrollable) มักใช้กับข้อมูลจำนวนมาก (คล้าย Column แต่จัดการเนื้อหาส่วนที่เกินขนาดหน้าจอได้ดีกว่า และสามารถเลื่อนได้)
- **GridView**: จัดวาง Widget ลูกในลักษณะของ ตาราง (Grid) ทั้งแนวตั้งและแนวนอน
- **Wrap**: คล้ายกับ Row หรือ Column แต่ถ้า Widget ลูกยาวเกินพื้นที่ จะขึ้นบรรทัดใหม่ให้โดยอัตโนมัติ เพื่อป้องกันหน้าจอ Error (Layout Overflow)
- **Flow**: ใช้สำหรับการจัดวาง Layout แบบกำหนดเองที่ซับซ้อนและเน้นประสิทธิภาพในการทำ Animation



# ทำความรู้จักกับ Row/Column widget



- **Row** (แนวนอน)

- ใช้สำหรับจัดวาง Widget ลูกเรียงกันจาก ซ้ายไปขวา
- Main Axis (แกนหลัก): แนวนอน (Horizontal)
- Cross Axis (แกนรอง): แนวตั้ง (Vertical)

- **Column** (แนวตั้ง)

- ใช้สำหรับจัดวาง Widget ลูกเรียงกันจาก บนลงล่าง
- Main Axis (แกนหลัก): แนวตั้ง (Vertical)
- Cross Axis (แกนรอง): แนวนอน (Horizontal)

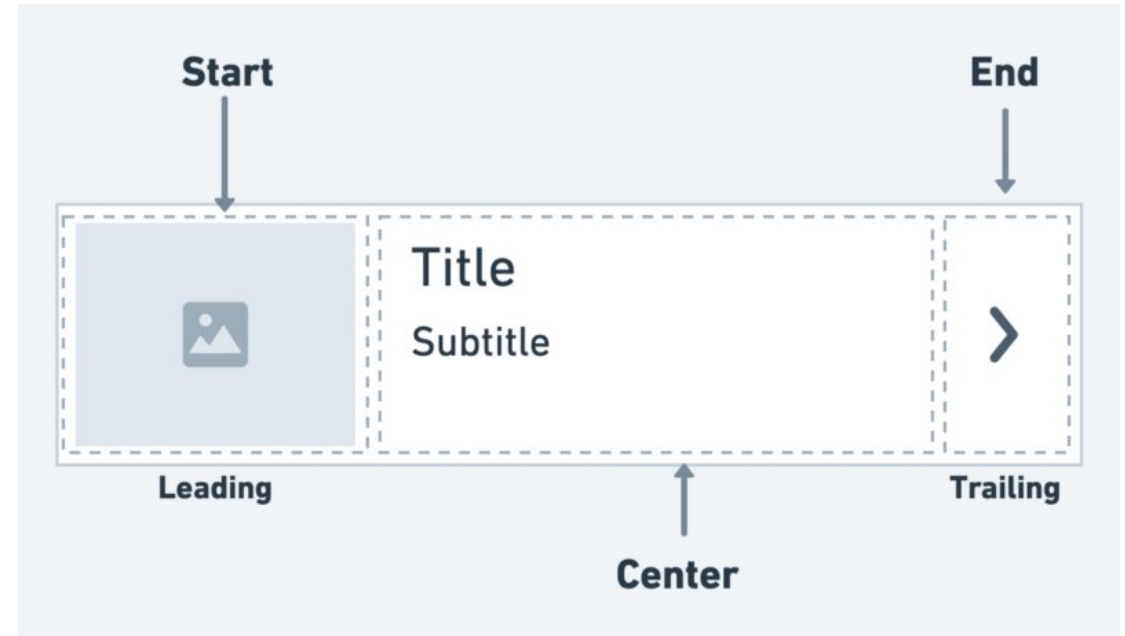
## ข้อควรระวัง!

ถ้าวาง Widget ใน Row หรือ Column จนยาวเกินขนาดหน้าจอ จะเกิดเส้นแถบสีเหลืองดำ (Layout Overflow) ขึ้นที่ เพราะจะ Scroll ไม่ได้ (ถ้าอยากให้ Scroll ได้ ต้องใช้ ListView หรือครอบด้วย SingleChildScrollView)

# ทำความรู้จักกับ ListTile widget

- **ListTile** คือ Widget สำเร็จรูปที่ออกแบบมาเพื่อใช้จัดการ แถวข้อมูล (Row) ในรายการ (List) โดยเฉพาะ
- โครงสร้างหลักของ ListTile จะแบ่งเป็น 4 ส่วนสำคัญ:
  - **leading**: วางไว้ หน้าสุด (ซ้ายมือ) มักใช้ใส่ Icon หรือ CircleAvatar (รูปโปรไฟล์)
  - **title**: ข้อความบรรทัดหลัก มักใช้เป็นหัวข้อตัวหนา
  - **subtitle**: ข้อความบรรทัดรอง อยู่ใต้ title มักใช้เป็นรายละเอียดเพิ่มเติม
  - **trailing**: วางไว้หลังสุด (ขวามือ) มักใช้เป็นปุ่มกด, ไอคอนลูกศร

มีฟังก์ชันรองรับการ "กด" มาให้ในตัว  
ทำให้จัดการการคลิกเลือกรายการได้ทันที



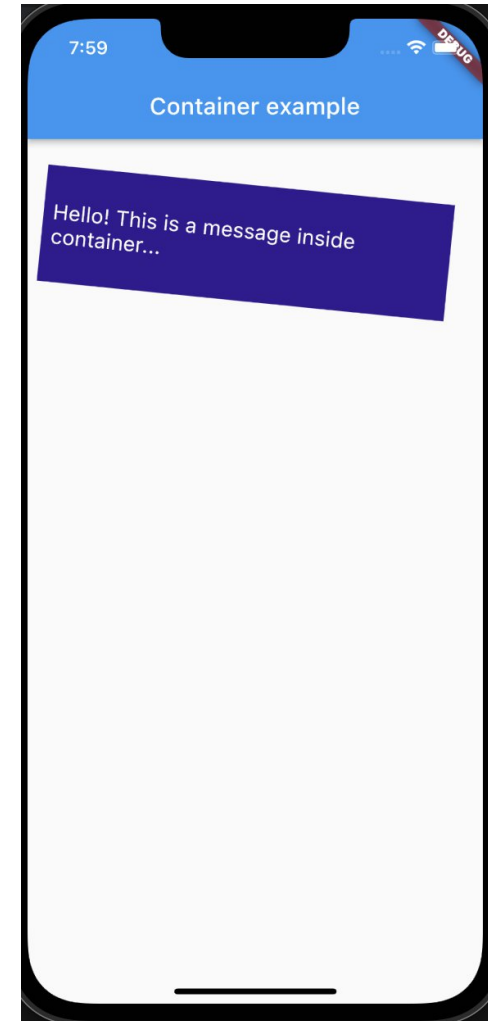
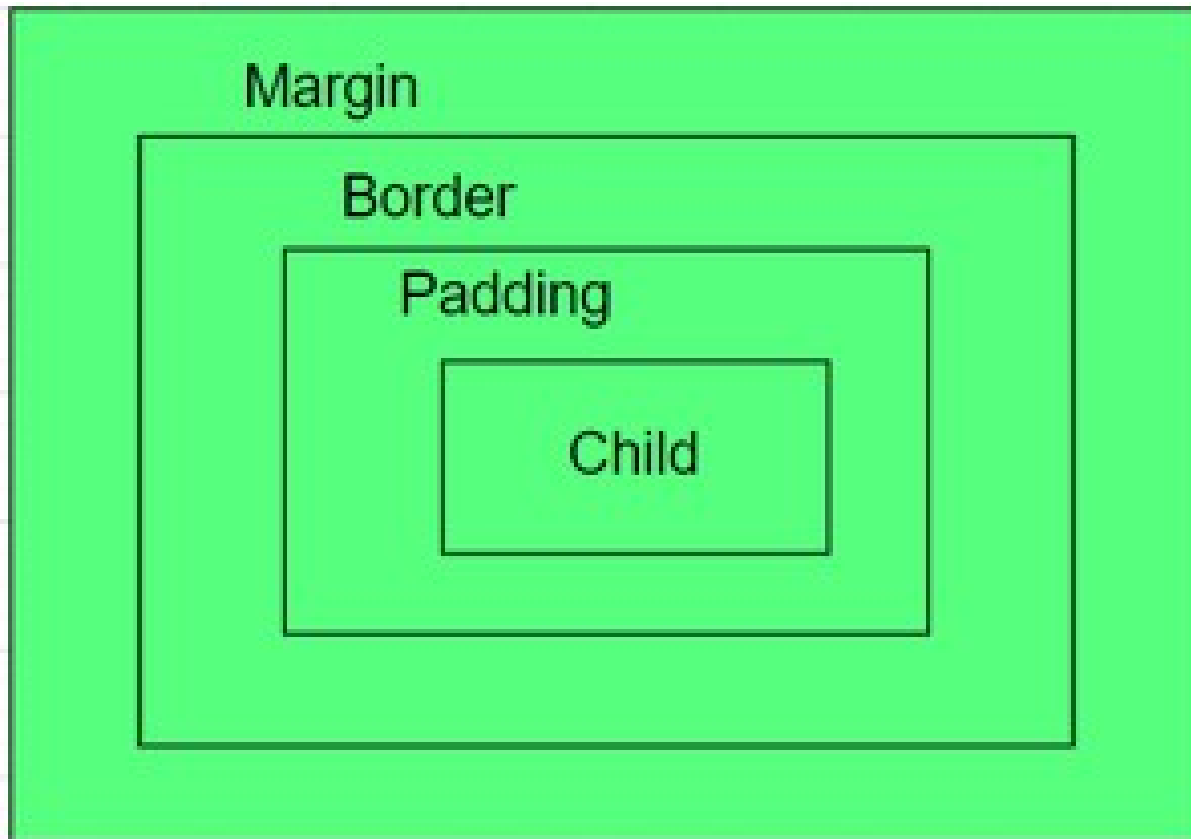
```
ListTile(  
  leading: Icon(Icons.car_rental),  
  title: Text('Car'),  
  trailing: Icon(Icons.more_vert),  
),
```

# ทดสอบ Basic Layouts

- สร้างโปรเจกต์ใหม่โดยใช้ชื่อ **basic\_layouts**
- นำตัวอย่างโค้ดจาก [ที่นี่](#)
- นำโค้ดตัวอย่างที่แตก ZIP เรียบร้อยแล้ว copy ทับลงในโปรเจกต์
- ภายในตัวอย่างจะประกอบด้วย ตัวอย่างการใช้งาน Container, ListView, GridView และ Stack

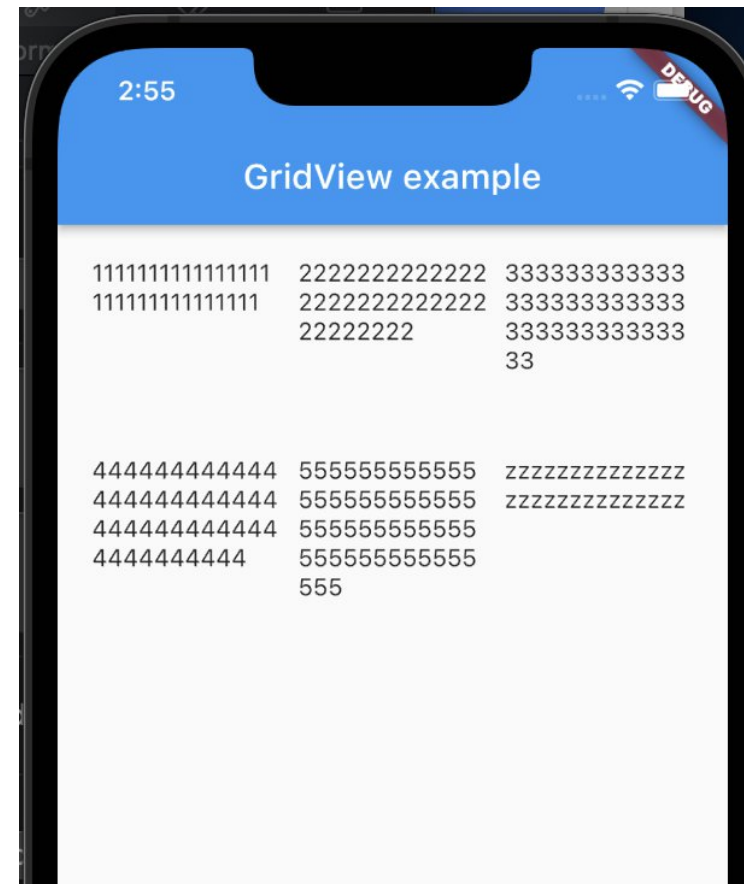
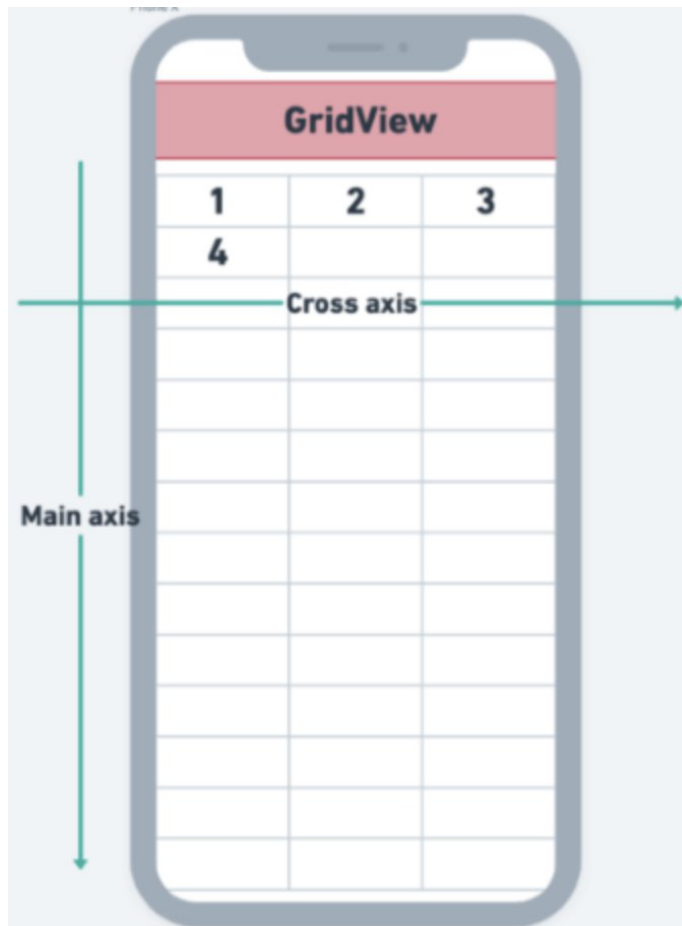
# ทดสอบ Basic Layouts (cont.)

- ตัวอย่างการใช้ **Container** Widget -  
Container is like a box to store contents



# ทดสอบ Basic Layouts (cont.)

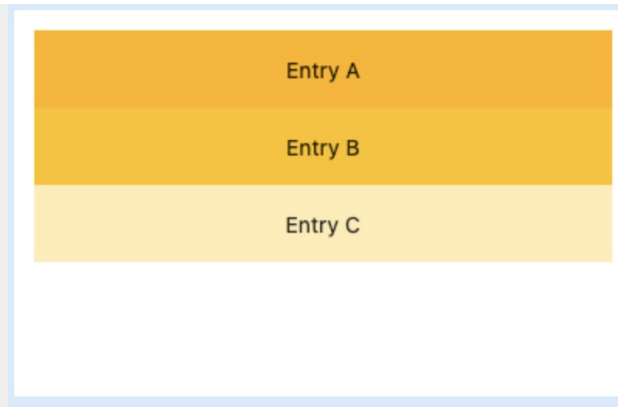
- ตัวอย่างการใช้ **GridView** Widget - GridView is a widget in Flutter that displays the items in a 2-D array (two-dimensional rows and columns)



# ทดสอบ Basic Layouts (cont.)

- ตัวอย่างการใช้ **ListView** Widget - ListView is used to group several items in an array and display them in a scrollable list

```
ListView(  
  padding: const EdgeInsets.all(8),  
  children: <Widget>[  
    Container(  
      height: 50,  
      color: Colors.amber[600],  
      child: const Center(child: Text('Entry A')),  
    ),  
    Container(  
      height: 50,  
      color: Colors.amber[500],  
      child: const Center(child: Text('Entry B')),  
    ),  
    Container(  
      height: 50,  
      color: Colors.amber[100],  
      child: const Center(child: Text('Entry C')),  
    ),  
  ],  
)
```

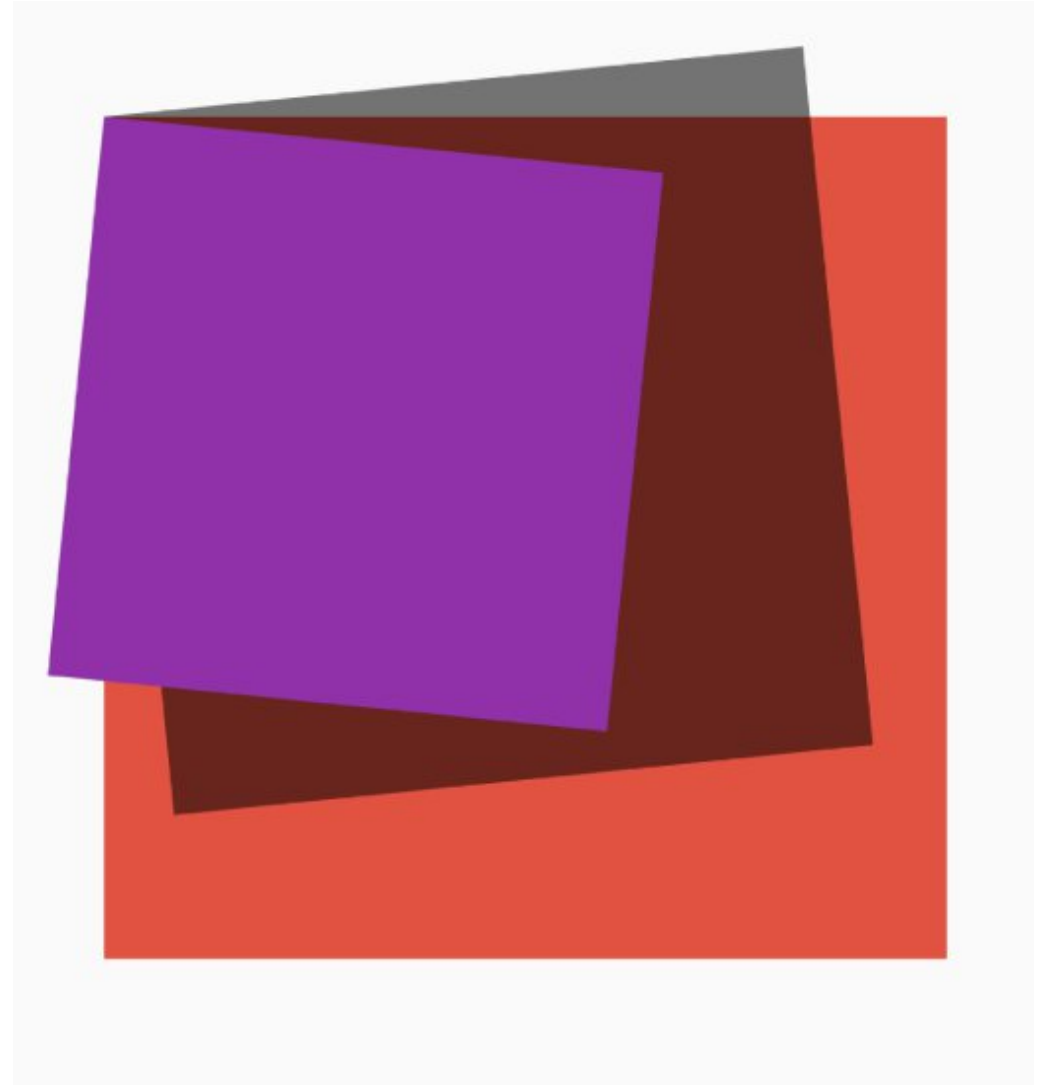


## ListView example

- |                                                                                     |                                      |   |
|-------------------------------------------------------------------------------------|--------------------------------------|---|
|  | Battery Full<br>The battery is full. | ★ |
|  | Anchor<br>Lower the anchor.          | ★ |
|  | Alarm<br>This is the time.           | ★ |

# ทดสอบ Basic Layouts (cont.)

- ตัวอย่างการใช้ **Stack** Widget -  
Stack widget is a built-in widget which allow to make a layer of widgets by putting them on the top of each other





# ทดสอบ Basic Layouts (cont.)

- การใช้ **SingleChildScrollView** - คือ Widget ที่ทำหน้าที่ทำให้พื้นที่การแสดงผล (Child) สามารถ "Scroll" หรือเลื่อนได้เมื่อเนื้อหาภายในมีขนาดใหญ่ เกินกว่าพื้นที่หน้าจอที่มีอยู่จริง
- เช่น หากวาง Widget หลายตัวใน Column จนยาวเกินหน้าจอ Flutter จะแสดงแถบสีเหลืองดำ (Overflow Warning) เพื่อเตือนว่าเนื้อหาหลุดขอบ การครอบด้วย SingleChildScrollView จะช่วยแก้ปัญหานี้ได้ เพราะส่วนที่เกินจะยังเลื่อนลงไปดูได้
- รองรับได้เพียงคนเดียว (Single Child) ใช้กับเนื้อหาที่ไม่ยาวมาก เหมาะสำหรับหน้า Form กรอกข้อมูล, หน้า Profile หรือหน้าที่มีเนื้อหาคงที่แต่ต้องการให้เลื่อนได้

# ทดสอบ Basic Layouts (cont.)

```
Widget build(BuildContext context) {  
  return MaterialApp(  
    home: Scaffold(  
      appBar: AppBar(title: const Text('SingleChildScrollView Demo')),  
      body: SingleChildScrollView(  
        // กำหนดทิศทางการเลื่อน (ปกติจะเป็นแนวตั้ง)  
        scrollDirection: Axis.vertical,  
        // เพิ่ม Padding รอบๆ เนื้อหาข้างใน  
        padding: const EdgeInsets.all(20),  
        child: Column(  
          children: [  
            const Text("เริ่มการเลื่อนดูเนื้อหา", style: TextStyle(fontSize: 24)),  
            const SizedBox(height: 20),  
            // สร้างกล่องสี่เหลี่ยมๆ กล่องให้ยาวเกินหน้าจอ  
            for (int i = 1; i <= 10; i++)  
              Container(  
                margin: const EdgeInsets.only(bottom: 10),  
                height: 150,  
                width: double.infinity,  
                color: Colors.blue[100 * (i % 9 + 1)],  
                child: Center(child: Text("กล่องที่ $i")),  
              ),  
            const Text("สิ้นสุดเนื้อหา"),  
          ],  
        ),  
      ),  
    ),  
  ),  
);
```

← **ครอบ Column อยู่**

**→**

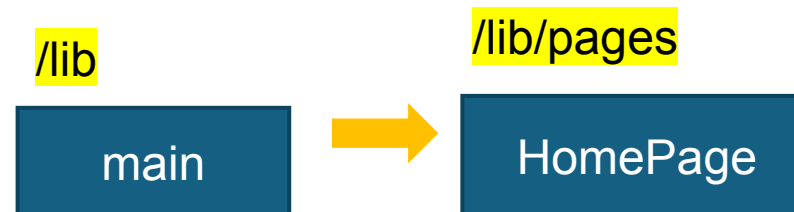
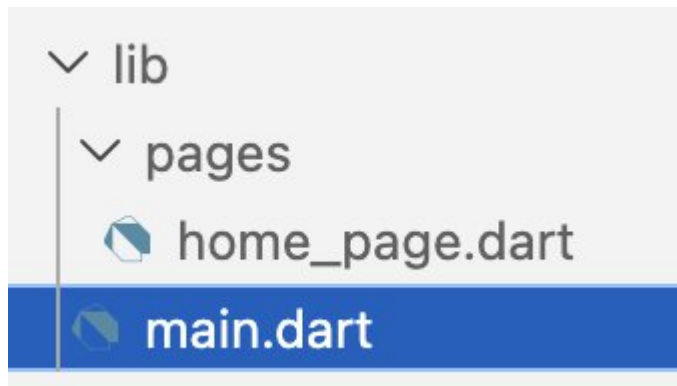
## ข้อจำกัด

- ห้ามวาง **Expanded** หรือ **Flexible** ไว้ใน **SingleChildScrollView** โดยตรง เพราะ Widget ในกลุ่ม Scroll จะมองว่าพื้นที่ภายในมีขนาดไม่จำกัดเลื่อนได้เรื่อยๆ ทำให้ **Expanded** ไม่สามารถคำนวณพื้นที่ที่เหลือได้และจะเกิด Error ทันที
- SingleChildScrollView** จะโหลด Widget ทั้งหมดที่อยู่ภายในขึ้นมาบน Memory ทันทีตั้งแต่เริ่มหน้าจอ หากมีข้อมูลเป็นจำนวนมากควรเปลี่ยนไปใช้ **ListView.builder** แทน เพราะจะโหลดเฉพาะรายการที่ปรากฏบนหน้าจอเท่านั้น

Q/A

# Basic Template (ตั้งต้น)

- นิสิตสามารถใช้ Basic Template ต่อไปนี้เป็นตัวตั้งต้นในการพัฒนา App ใดๆ ได้ต่อจากนี้ โดยหลังจากสร้างโปรเจกต์เสร็จเรียบร้อยแล้ว สามารถ Download รูปแบบเบื้องต้นจาก [ที่นี่](#)
- เมื่อได้ไฟล์ **lib\_basic\_template.zip** มาแล้วให้แตกไฟล์ออก จากนั้นให้นำโฟลเดอร์ lib ที่ได้ไปวางทับโฟลเดอร์ lib ของโปรเจกต์ที่สร้างใหม่
- โครงสร้างเบื้องต้นมีดังนี้ นิสิตสามารถแก้ไขเพิ่มเติมจากพื้นฐานนี้ได้ทันที



# การบ้าน #1

- สร้าง New App ชื่อว่า **my\_profile** เพื่อแสดงข้อมูล Profile ของตนเอง โดยให้ App มีเพียง 1 หน้าเท่านั้น
- ภายในหน้า Profile จะต้องประกอบด้วย
  - ภาพแทนตัวนิสิต
  - รหัสนิสิต, ชื่อ, นามสกุล
  - ส่วนแสดงข้อมูลการติดต่อ เช่น Email, Phone หรือช่องทาง Social Network
  - กำหนดให้ต้องมี Icon ประกอบคู่กับข้อความ
- เมื่อเนื้อหาแสดงเกินความยาวของหน้าจอ จะต้องสามารถขยับเลื่อนได้ (Scrollable)

\* ส่งในชั่วโมงเรียนคาบถัดไป